DATA STRUCTURES

Complete Notes: Basics to Advanced

Concepts • Operations • Complexity • Algorithms • Interview Preparation

Designed for: beginners, computer science students, competitive programmers, and coding interview preparation.

Includes: foundational concepts, arrays, linked lists, stacks, queues, hashing, trees, balanced trees, heaps, graphs, tries, union-find, sorting, searching, recursion, backtracking, greedy methods, dynamic programming, and advanced design patterns.

Language-neutral notes with pseudocode. Implement examples in C, C++, Java, C#, Python, or another language.

How to Study This Guide

- Read the concept first, then trace each operation by hand.
- Implement each structure in one programming language; test empty, one-item, duplicate, sorted, reverse-sorted, and large inputs.
- Memorize the reason for each complexity rather than only memorizing a table.
- For interviews, explain the invariant, edge cases, time complexity, and space complexity before coding.

Recommended sequence

Stage	Topics
1. Foundations	Data structures, ADTs, Big-O, arrays, strings
2. Linear structures	Linked lists, stacks, queues, deques, hashing
3. Hierarchical structures	Trees, BSTs, AVL/red-black concepts, heaps, tries
4. Networks and sets	Graphs, BFS/DFS, shortest paths, MST, DSU
5. Algorithms	Searching, sorting, recursion, backtracking, greedy, dynamic programming
6. Mastery	Complexity proofs, testing, design choices, interview practice

Contents

- 1. Data Structure Foundations
- 2. Complexity Analysis
- 3. Arrays and Dynamic Arrays
- 4. Strings
- 5. Linked Lists
- 6. Stacks
- 7. Queues and Deques
- 8. Hashing and Hash Tables
- 9. Trees and Traversals
- 10. Binary Search Trees and Balanced Trees
- 11. Heaps and Priority Queues
- 12. Tries and Prefix Structures
- 13. Graphs
- 14. Disjoint Set Union
- 15. Searching
- 16. Sorting
- 17. Recursion and Backtracking
- 18. Greedy Algorithms
- 19. Dynamic Programming
- 20. Advanced Data Structures Overview
- 21. Choosing a Structure and Complexity Tables
- 22. Testing, Debugging, and Common Mistakes
- 23. Interview Questions
- 24. Practice Set and Projects

1. Data Structure Foundations

A data structure organizes information so operations can be performed efficiently and correctly.

1.1 Essential terminology

- **Data structure:** a representation for storing and accessing data.
- **Algorithm:** a finite sequence of steps that transforms input into output.
- **ADT (Abstract Data Type):** defines behavior and operations, not the internal implementation. A stack ADT exposes push, pop, and peek.
- **Invariant:** a property that must remain true, such as all values in a min-heap being no smaller than their children.
- **Amortized analysis:** average cost per operation over a sequence, even if an occasional operation is expensive.

1.2 Classification

Type	Description	Examples
Primitive	Basic built-in values	integer, Boolean, character, float
Linear	Elements form a sequence	array, list, stack, queue
Non-linear	Hierarchical/network relationships	tree, graph
Static capacity	Capacity fixed by representation	fixed-size array
Dynamic capacity	Can grow or shrink	dynamic array, hash table
Associative	Values accessed by key	map/dictionary, set

1.3 Common operations

Access, search, insert, delete, update, traverse, sort, merge, and test membership. The best structure depends on which operations are frequent and which constraints matter: memory, ordering, duplicates, predictable latency, and concurrency.

1.4 Correctness before speed

An efficient algorithm is useful only if it is correct. State preconditions, postconditions, and invariants. For example, binary search requires sorted data; using it on unsorted data can return incorrect results.

2. Complexity Analysis

Complexity describes how time or memory grows as input size n increases.

2.1 Asymptotic notation

- **O(f(n)):** asymptotic upper bound.
- **Ω(f(n)):** asymptotic lower bound.
- **Θ(f(n)):** tight bound when upper and lower bounds match.
- Drop constants and lower-order terms: $7n^2 + 3n + 20$ is $\Theta(n^2)$.

Complexity	Name	Example
O(1)	Constant	array access by index
O(log n)	Logarithmic	binary search
O(n)	Linear	scan all elements
O(n log n)	Linearithmic	merge sort, heap sort
$O(n^2)$	Quadratic	all pairs in a simple nested loop
O(2■)	Exponential	enumerating all subsets
O(n!)	Factorial	enumerating all permutations

2.2 How to analyze code

- Sequential operations: add costs, then keep the dominant term.
- Nested loops: multiply iteration counts when both depend on n.
- Loop that halves or doubles a value each iteration: often O(log n).

- Recursion: write a recurrence such as T(n)=2T(n/2)+O(n), which gives O(n log n) for merge sort.
- Space: count auxiliary arrays, recursion stack, nodes, and temporary structures.

```
for i = 1 to n:          // O(n)
    print(i)

for i = 1 to n:          // O(n^2)
    for j = 1 to n:
        print(i, j)

while n > 1:             // O(log n)
    n = floor(n / 2)
```

2.3 Worst, average, and amortized cases

Worst-case complexity gives an upper bound for any input of size n. Average-case complexity depends on an input distribution. Amortized analysis bounds a sequence of operations—for example, appending to a dynamic array is O(1) amortized, although resizing one append can cost O(n).

2.4 Trade-offs

Time and space often trade off: a hash table spends extra memory to make average lookup fast; memoization stores results to avoid repeated computation; an in-place algorithm reduces extra memory but may be harder to implement.

3. Arrays and Dynamic Arrays

3.1 Array basics

An array stores elements in contiguous indexed positions. If elements have fixed size, the address of index i can be calculated directly, enabling O(1) random access.

3.2 Complexity

Operation	Typical cost	Reason
Read/update by index	O(1)	Direct index calculation
Search unsorted array	O(n)	May inspect every item
Insert/delete at end	O(1) amortized for dynamic array	Occasional resize
Insert/delete at front or middle	O(n)	Elements shift
Traverse	O(n)	Visit each item

3.3 Dynamic array

A dynamic array tracks size and capacity. When full, it allocates a larger buffer, copies existing elements, and then inserts the new value. Geometric growth (often doubling) makes append O(1) amortized.

- Size = number of stored elements; capacity = available slots.
- Reserve capacity when a good estimate is known to reduce reallocations.
- Resizing can invalidate pointers/references in languages where storage moves.
- Deleting an element may shift later elements; clearing a slot is not always enough.

3.4 Common patterns

- **Two pointers:** move indices from ends or maintain a moving window.
- **Sliding window:** update a running property as the window moves, often reducing $O(n^2)$ to O(n).
- **Prefix sums:** P[i] stores sum of the first i values; range sum [l,r) = P[r] – P[l].
- **Difference array:** supports efficient range increments, followed by a prefix-sum reconstruction.

```
prefix[0] = 0
for i = 0 to n-1:
    prefix[i+1] = prefix[i] + a[i]
# sum from l inclusive to r exclusive:
range_sum = prefix[r] - prefix[l]
```

4. Strings

A string is a sequence of characters or code units. Representation differs by language: UTF-8 uses variable-length bytes; UTF-16 uses code units; a user-perceived character may contain multiple code points.

4.1 Important concepts

- Strings are immutable in many languages; repeated concatenation in a loop can be costly. Use a builder/buffer when appropriate.
- Be careful with empty strings, Unicode, case folding, whitespace, and locale-dependent comparisons.
- Substring/search costs depend on language and representation; copying a substring may require O(k) time and space.

4.2 Common algorithms

- **Naive pattern search:** compare pattern at every candidate start; worst-case O(nm).
- **KMP:** uses a prefix/failure table to avoid repeating comparisons; O(n+m).
- **Rolling hash:** compares fingerprints of windows; fast in practice, but hash collisions must be considered.
- **Trie:** stores prefixes to support dictionary lookup and autocomplete.

```
reverse(s):
    left = 0; right = length(s) - 1
    while left < right:
        swap(s[left], s[right])
        left += 1; right -= 1
```

4.3 Palindrome check

Compare characters from both ends and move inward. Time O(n), extra space O(1) if working with an indexable sequence and ignoring normalization costs.

5. Linked Lists

A linked list stores nodes connected by references/pointers. A singly linked node typically contains a value and a next pointer; a doubly linked node also has prev.

5.1 Variants

- Singly linked list: next pointer only.
- Doubly linked list: next and previous pointers; easier deletion when node is known, but uses extra memory.
- Circular list: last node points back to the first; useful for cyclic traversal.

5.2 Complexity

Operation	Singly linked list
Access by position	O(n)
Search	O(n)
Insert at head	O(1)
Delete head	O(1)
Insert after a known node	O(1)
Delete a known node	O(1) only if required links are available; singly linked deletion usually needs predecessor

5.3 Core operations

```
push_front(value):
    node = new Node(value)
    node.next = head
    head = node

reverse(head):
    prev = null
    cur = head
    while cur != null:
        next_node = cur.next
        cur.next = prev
        prev = cur
        cur = next_node
    return prev
```

5.4 Fast/slow pointer patterns

- Middle node: slow advances one step, fast advances two.

- Cycle detection: if fast and slow meet, a cycle exists (Floyd's algorithm).
- Find cycle entry: after a meeting, reset one pointer to head; advance both one step at a time.
- Nth node from end: keep two pointers n positions apart, then move together.

5.5 Pitfalls

Update links carefully, avoid losing the remainder of a list, handle empty and single-node lists, and update head/tail correctly. In manual-memory languages, also manage allocation and deallocation.

6. Stacks

A stack is a LIFO structure: last in, first out.

6.1 Operations and complexity

Operation	Meaning	Typical cost
push(x)	Add to top	O(1)
pop()	Remove and return top	O(1)
peek/top()	Read top	O(1)
isEmpty()	Check empty	O(1)

6.2 Implementations

Use a dynamic array for cache-friendly storage and simple indexing, or a linked list for node-based growth. A stack must define what happens on empty pop/peek: error, exception, optional result, or sentinel.

6.3 Applications

- Function calls and recursion call stacks.
- Balanced parentheses and expression parsing.
- Undo/redo and browser history models.
- Depth-first search and backtracking.
- Monotonic stack for next greater/smaller element and histogram problems.

```
is_balanced(text):
    stack = empty
    for ch in text:
        if ch is an opening bracket:
            stack.push(ch)
        else if ch is a closing bracket:
            if stack is empty or stack.pop() does not match ch:
                return false
    return stack.is_empty()
```

7. Queues and Deques

A queue is FIFO: first in, first out. A deque (double-ended queue) supports insertion and removal at both ends.

7.1 Complexity

Operation	Queue / deque typical cost
Enqueue at rear	O(1)
Dequeue at front	O(1)
Peek front/rear	O(1)
Search arbitrary value	O(n)

7.2 Circular array queue

Maintain head, tail, size, and capacity. Advance an index using (index + 1) modulo capacity. Tracking size or reserving one unused slot distinguishes full from empty.

7.3 Applications

- Breadth-first search (BFS).

- Task scheduling, producer-consumer buffers, and request handling.
- Sliding-window maximum using a monotonic deque.
- Round-robin scheduling using a circular queue.

7.4 Priority queue is different

A priority queue removes an element based on priority, not arrival order. A heap is a common implementation; it is covered in Chapter 11.

8. Hashing and Hash Tables

A hash table maps keys to buckets using a hash function. A good hash distributes keys broadly; equality must be consistent with hashing.

8.1 Collision resolution

- **Separate chaining:** each bucket holds a collection of entries.
- **Open addressing:** entries occupy slots in the table; collision strategies include linear probing, quadratic probing, and double hashing.
- Deletion in open addressing needs tombstones or another strategy so searches do not stop too early.

8.2 Load factor

Load factor α = number of stored entries / number of buckets. As load grows, collisions generally increase. Resizing and rehashing are used to restore a target load factor.

8.3 Complexity and caveats

Operation	Average expected	Worst case
Search	O(1)	O(n)
Insert	O(1) amortized	O(n) during resize or severe collisions
Delete	O(1) average	O(n)

- Hash tables do not inherently maintain sorted order.
- Do not mutate a key in a way that changes its hash while it is stored.
- Use a set for membership and a map/dictionary for key-value association.
- Security-sensitive systems may need hash functions resistant to adversarial collision attacks.

9. Trees and Tree Traversals

A tree is a connected acyclic hierarchical structure. Terms include root, parent, child, sibling, leaf, depth, height, subtree, and degree.

9.1 Binary tree types

- Full/proper: every node has either zero or two children.
- Complete: every level is full except possibly the last, which fills left to right.
- Perfect: all internal nodes have two children and all leaves share a depth.
- Balanced: height is kept within a bound, often O(log n), according to a specific rule.

9.2 Traversals

Traversal	Order	Typical use
Preorder	Node, left, right	Copy/serialize a tree
Inorder	Left, node, right	Sorted order in a BST
Postorder	Left, right, node	Delete/free subtrees
Level order	Breadth-first by level	Level-wise processing

```
preorder(node):
    if node == null: return
    visit(node)
    preorder(node.left)
```

```
    preorder(node.right)
```

9.3 Complexity

A traversal visits each node once: O(n) time. Recursive traversal uses O(h) call-stack space, where h is tree height; h is O(log n) for a balanced tree and O(n) for a skewed tree.

10. Binary Search Trees and Balanced Trees

10.1 Binary search tree (BST)

For each node, keys in the left subtree are smaller and keys in the right subtree are larger, according to the chosen duplicate-key policy. Search, insert, and delete take O(h), where h is height.

10.2 BST deletion cases

- Leaf: remove it.
- One child: replace node with its child.
- Two children: replace its key with the inorder successor (minimum in right subtree) or predecessor, then remove that replacement node.

10.3 Balanced search trees

Structure	Main idea	Typical search/insert/delete
AVL tree	Strict height balance using balance factors and rotations	O(log n)
Red-black tree	Color rules constrain height; fewer strict balancing requirements	O(log n)
B-tree / B+ tree	Multiway balanced tree optimized for block/disk access	O(log n) tree height
Treap	BST order by key and heap order by randomized priority	Expected O(log n)

10.4 Rotations

Left and right rotations locally restructure a tree while preserving BST order. AVL balance factor is height(left) − height(right); imbalance is repaired with single or double rotations. Red-black trees use recoloring and rotations to preserve their invariants.

10.5 When to use

Use a balanced search tree when sorted iteration, range queries, predecessor/successor, and predictable O(log n) updates matter. Hash maps are often simpler for average O(1) exact-key lookup but do not naturally support ordered range traversal.

11. Heaps and Priority Queues

A binary heap is a complete binary tree commonly stored in an array. In a min-heap each parent is no greater than its children; a max-heap reverses the order.

11.1 Array index relationships

```
For zero-based indexing:
left(i)  = 2*i + 1
right(i) = 2*i + 2
parent(i)= floor((i-1)/2)
```

11.2 Operations

Operation	Binary heap cost
Peek min/max	O(1)
Insert	O(log n)
Remove top	O(log n)
Build heap from n items	O(n)
Find arbitrary value	O(n)

11.3 Sift up/down

After insertion, sift the new item upward while it violates the heap property. After removing the root, move the last item to the root and sift downward. Bottom-up heap construction is O(n), not O(n log n), because most nodes are near the leaves.

11.4 Applications

- Scheduling by priority.
- Dijkstra's algorithm and other graph algorithms.
- Top-k largest/smallest elements.
- Heap sort: O(n log n) time and O(1) auxiliary array space for the in-place form.

12. Tries and Prefix Structures

A trie stores strings by characters or tokens along paths. Each edge represents a symbol; a terminal marker indicates a complete key.

12.1 Operations

- Insert a word: O(L), where L is word length.
- Search a word: O(L).
- Check whether a prefix exists: O(L).
- Space depends on the number of nodes and alphabet representation.

12.2 Variants

A basic trie may use an array of children for a small fixed alphabet or a map for sparse alphabets. A compressed/radix trie merges single-child paths. A ternary search trie uses low/equal/high links.

12.3 Applications

- Autocomplete and prefix search.
- Dictionary lookup and spell-checking support.
- IP routing via bitwise prefix tries.
- Word search and prefix-based filtering.

> Tries can use more memory than hash tables for exact lookups, but they support prefix operations naturally.

13. Graphs

A graph G=(V,E) contains vertices V and edges E. Graphs may be directed/undirected, weighted/unweighted, cyclic/acyclic, connected/disconnected, and sparse/dense.

13.1 Representations

Representation	Space	Best fit
Adjacency list	O(V+E)	Sparse graphs; iterate neighbors
Adjacency matrix	$O(V^2)$	Dense graphs; O(1) edge test
Edge list	O(E)	Algorithms that process all edges

13.2 BFS and DFS

BFS uses a queue and explores by distance in number of edges; it finds shortest paths in unweighted graphs. DFS uses recursion or an explicit stack and is useful for connectivity, cycle detection, topological sorting, and component discovery.

```
BFS(start):
    mark start visited; queue.push(start)
    while queue not empty:
        v = queue.pop_front()
        for each neighbor u of v:
            if u not visited:
                mark u visited; parent[u] = v
                queue.push_back(u)
```

13.3 Important graph algorithms

Problem	Algorithm	Typical complexity
Unweighted shortest path	BFS	O(V+E)
Shortest path, nonnegative weights	Dijkstra + priority queue	O((V+E) log V) with common heap implementation
Negative edges, detect reachable negative cycle	Bellman–Ford	O(VE)
All-pairs shortest paths	Floyd–Warshall	$O(V^3)$ time, $O(V^2)$ space
Minimum spanning tree	Kruskal + DSU	O(E log E)
Minimum spanning tree	Prim	Common heap version O(E log V)
DAG ordering	Topological sort	O(V+E)

13.4 Topological sort

A topological ordering exists only for a directed acyclic graph (DAG). Kahn's algorithm repeatedly removes zero-indegree vertices; if not all vertices are emitted, the graph has a cycle. DFS finishing order is another approach.

13.5 Strongly connected components

In a directed graph, an SCC is a maximal set of vertices where every vertex can reach every other. Kosaraju's and Tarjan's algorithms find SCCs in O(V+E).

14. Disjoint Set Union (Union-Find)

DSU maintains a partition of elements into disjoint sets, supporting find(x) and union(a,b).

14.1 Optimizations

- Path compression shortens paths during find.
- Union by size/rank attaches the smaller/shallower tree under the larger/deeper tree.
- Together, operations run in near-constant amortized time, $O(\alpha(n))$ per operation, where α is the inverse Ackermann function.

```
find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])  # path compression
    return parent[x]

union(a, b):
    ra = find(a); rb = find(b)
    if ra == rb: return
    # attach lower-rank root below higher-rank root
```

14.2 Uses

- Kruskal minimum spanning tree.
- Dynamic connectivity when edges are added.
- Grouping equivalent items.
- Detecting whether an undirected edge connects vertices already in the same component.

Classic DSU does not efficiently support arbitrary deletion of edges or splitting a set.

15. Searching Algorithms

Algorithm	Requirement	Time	Extra space
Linear search	None	O(n)	O(1)
Binary search	Sorted random-access sequence	O(log n)	O(1) iterative
Hash lookup	Hash/equality operations	O(1) average	Table-dependent
BST search	Search tree	O(h)	O(h) recursive
Trie search	Key/prefix sequence	O(L)	Structure-dependent

15.1 Binary search

```
binary_search(a, target):
    lo = 0; hi = length(a)  # half-open [lo, hi)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if a[mid] < target: lo = mid + 1
        else: hi = mid
    return lo  # insertion point / first index >= target
```

This lower-bound variant returns the first index whose value is at least target. Check whether that index is in range and equal to target to test membership. Half-open ranges reduce off-by-one errors.

15.2 Binary search on an answer

If a monotonic predicate changes from false to true (or true to false), binary search can find the boundary. Examples include minimum capacity sufficient for a workload or the smallest feasible rate. Correctness depends on proving monotonicity.

16. Sorting Algorithms

Algorithm	Best	Average	Worst	Stable?
Bubble sort	O(n) with early exit	O(n²)	O(n²)	Yes
Selection sort	O(n²)	O(n²)	O(n²)	Usually no
Insertion sort	O(n)	O(n²)	O(n²)	Yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	Yes
Quicksort	O(n log n)	O(n log n)	O(n²)	Usually no
Heap sort	O(n log n)	O(n log n)	O(n log n)	No
Counting sort	O(n+k)	O(n+k)	O(n+k)	Can be
Radix sort	O(d(n+k))	O(d(n+k))	O(d(n+k))	Typically yes

16.1 Merge sort

Divide the array into halves, recursively sort each half, and merge. It guarantees O(n log n) time and typically uses O(n) auxiliary memory for arrays.

16.2 Quicksort

Choose a pivot, partition elements, then recursively sort partitions. Randomized pivots or good partition strategies reduce the chance of bad inputs, but worst-case time remains O(n²) for common implementations.

16.3 Stability and in-place

A stable sort preserves the relative order of equal-key elements. In-place describes low extra storage, but exact definitions vary. Select a sort based on data size, stability needs, memory limits, existing order, and library guarantees.

16.4 Non-comparison sorting

Counting and radix sorts can beat O(n log n) comparison-sorting bounds when keys have suitable bounded or structured representations. Their time depends on key range, number of digits, and auxiliary memory.

17. Recursion and Backtracking

17.1 Recursion

A recursive function solves a problem by solving smaller instances. Every recursive design needs a base case and progress toward it. The call stack uses memory proportional to recursion depth.

```
factorial(n):
    if n <= 1: return 1
    return n * factorial(n - 1)
```

17.2 Backtracking

Backtracking explores a decision tree, makes a choice, recursively explores, and undoes the choice. It is used for permutations, combinations, Sudoku, N-Queens, and constraint search.

```
search(state):
    if complete(state):
        record(state); return
    for choice in valid_choices(state):
```

```
        apply(choice, state)
        search(state)
        undo(choice, state)
```

17.3 Pruning

Pruning rejects a partial state when no valid solution can extend it. Correct pruning must not remove any valid solution. Estimate complexity from the branching factor and depth; many backtracking problems are exponential.

18. Greedy Algorithms

A greedy algorithm chooses the locally best-looking option at each step. Greedy choices are correct only when supported by a proof, often an exchange argument, cut property, or stays-ahead argument.

18.1 Common examples

- Interval scheduling: repeatedly choose the compatible interval with earliest finishing time for maximum number of non-overlapping intervals.
- Kruskal and Prim: minimum spanning tree algorithms use the cut property.
- Huffman coding: repeatedly combine two least-frequent trees.
- Dijkstra: greedily finalizes the nearest unsettled vertex when edge weights are nonnegative.

18.2 When greedy fails

A locally optimal choice can block a globally optimal solution. For coin change, a largest-coin-first strategy works for some denominations but not all. If no proof is apparent, compare with dynamic programming or exhaustive search on small examples.

19. Dynamic Programming

Dynamic programming (DP) solves overlapping subproblems and reuses results. It is useful when a problem has optimal substructure and a manageable number of distinct states.

19.1 Two styles

- **Memoization:** top-down recursion with cached results.
- **Tabulation:** bottom-up table filled in dependency order.
- State defines what a subproblem means; transition defines how to compute it; base cases anchor the recurrence; answer extraction identifies the required result.

19.2 DP design checklist

- Define the state precisely, including indices, capacities, or remaining choices.
- Write the recurrence and prove it covers all choices.
- Set base cases and establish computation order.
- Count states × work per state for time; count stored states for space.
- Optimize space only after correctness is established.

19.3 Example: Fibonacci

```
fib(n):
    if n <= 1: return n
    dp[0] = 0; dp[1] = 1
    for i = 2 to n:
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
# Time O(n); table space O(n), reducible to O(1).
```

19.4 Common DP families

Family	Typical state / example
1D DP	dp[i] based on earlier positions; climbing stairs
Grid DP	dp[row][col]; paths in a grid
Knapsack	dp[item][capacity] or compressed capacity dimension
Subsequence	LIS, LCS, edit distance
Interval DP	Best answer for subarray/interval [l,r]
Tree DP	Best result for a node and child states
Bitmask DP	State includes a subset of up to roughly 20–25 items, depending on constraints
Digit DP	Count numbers satisfying digit constraints

19.5 Complexity example

For 0/1 knapsack with n items and integer capacity W, the standard DP is O(nW) time and O(nW) space, often reduced to O(W). This is pseudo-polynomial because it depends on numeric W, not only the number of bits needed to represent W.

20. Advanced Data Structures Overview

20.1 Fenwick tree (Binary Indexed Tree)

Supports point updates and prefix-sum queries in O(log n), using O(n) space. A range sum can be computed from two prefix sums. It is compact and simpler than a segment tree, but standard forms are best suited to invertible prefix aggregates such as sums.

20.2 Segment tree

Stores summaries of intervals in a tree. Build is usually O(n); point update and range query are O(log n). Lazy propagation supports selected range updates in O(log n), when the aggregate and update operations can be composed correctly.

20.3 Sparse table

For static data and idempotent operations such as min, max, or gcd, preprocessing takes O(n log n) and range queries can be O(1). It is not designed for arbitrary point updates.

20.4 Monotonic stack / queue

Maintains elements in increasing or decreasing order while processing a sequence. Each element is inserted and removed at most once, so common next-greater and sliding-window extrema algorithms run in O(n).

20.5 LRU cache

A common O(1)-average LRU cache combines a hash map (key to node) with a doubly linked list ordered from most-recent to least-recent. On access, move a node to the front; when full, evict the tail.

20.6 Bloom filter

A probabilistic set-membership structure using a bit array and multiple hashes. It can report false positives but, in its standard form, not false negatives for inserted items. It uses compact memory when occasional false positives are acceptable.

20.7 Persistent and immutable structures

Persistent structures preserve prior versions after updates, often using structural sharing. They are useful for undo histories, functional programming, snapshots, and versioned data.

20.8 External-memory structures

B-trees and B+ trees reduce disk/page accesses by storing multiple keys per node. They are widely used in databases and filesystems because storage I/O cost often dominates CPU cost.

20.9 Choosing an advanced structure

Start from the query/update pattern, required guarantees, memory budget, and whether data changes. Advanced structures add implementation complexity; use them when their operation guarantees solve a real bottleneck.

21. Choosing a Structure and Complexity Tables

Need	Likely structure	Why
Fast indexed access	Array / dynamic array	O(1) index access
Fast average exact-key lookup	Hash map / set	Expected O(1) lookup
Sorted keys and range queries	Balanced search tree	O(log n) updates and ordered traversal
Frequent minimum/maximum removal	Heap / priority queue	O(log n) insertion/removal
FIFO processing	Queue	O(1) end operations
LIFO processing / undo	Stack	O(1) top operations
Prefix search	Trie	Cost based on key length
Network relationships	Graph	Explicit vertices and edges
Connectivity under additions	DSU	Near-constant amortized union/find
Range aggregates with updates	Fenwick / segment tree	O(log n) common operations
Disk-based ordered index	B-tree / B+ tree	Minimizes page access

21.1 Core complexity summary

Structure	Access	Search	Insert	Delete
Array	O(1) index	O(n)	O(n) middle	O(n) middle
Linked list	O(n)	O(n)	O(1) at known position	O(1) if links available
Hash table	—	O(1) average	O(1) average amortized	O(1) average
Balanced BST	O(log n) by key	O(log n)	O(log n)	O(log n)
Binary heap	O(1) top	O(n) arbitrary	O(log n)	O(log n) top removal
Trie	By prefix	O(L)	O(L)	O(L) plus cleanup

These are typical bounds under stated assumptions. Hash tables have worst-case O(n) operations; ordinary BSTs can become linear-height; linked-list insertion is O(1) only when the relevant node/link is already known.

22. Testing, Debugging, and Common Mistakes

22.1 Edge-case checklist

- Empty input, one item, all equal values, duplicates, negative values, and extreme numeric values.
- Already sorted and reverse-sorted inputs.
- Missing key, key at first/last position, and values outside the range.
- Cycles or disconnected components for graph/list code.
- Overflow, invalid indices, null references, and recursion depth.

22.2 Frequent mistakes

- Using binary search on unsorted data.
- Off-by-one errors in loops and half-open ranges.
- Forgetting to update head, tail, size, parent, or visited state.
- Assuming a hash table preserves sorted order.
- Using Dijkstra with negative edge weights.
- Claiming O(1) for a linked-list operation that first requires O(n) traversal.
- Confusing a heap with a fully sorted array.
- Writing a DP recurrence without defining the state or base cases.

22.3 Prove and measure

Use invariants for correctness, complexity analysis for scaling, and benchmarks for real performance. Benchmark representative inputs; account for allocation, cache locality, and constant factors. Do not replace a simpler correct solution with a complex structure without evidence.

23. Interview Questions and Short Answers

Q: What is an ADT?
A: A specification of values and operations independent of implementation.

Q: Array vs linked list?
A: Arrays offer O(1) indexed access and locality; linked lists offer O(1) insertion/deletion at a known position but O(n) access.

Q: Why is dynamic-array append amortized O(1)?
A: Geometric capacity growth spreads occasional O(n) copying over many cheap appends.

Q: Stack vs queue?
A: Stack is LIFO; queue is FIFO.

Q: How does a hash table handle collisions?
A: Commonly separate chaining or open addressing.

Q: BST vs heap?
A: BST supports ordered search and traversal; heap efficiently exposes the min/max but does not support fast arbitrary search.

Q: Why balance a search tree?
A: To keep height O(log n), giving predictable logarithmic operations.

Q: BFS vs DFS?
A: BFS explores by edge distance and finds unweighted shortest paths; DFS explores deeply and supports cycle/SCC/topological techniques.

Q: When is topological sorting possible?
A: For a directed acyclic graph.

Q: What does DSU solve?
A: Maintains disjoint components with efficient find and union.

Q: Stable sort?
A: Equal-key items retain their relative input order.

Q: What is a greedy proof?
A: A correctness argument showing local choices can be part of a global optimum.

Q: Memoization vs tabulation?
A: Memoization is top-down caching; tabulation is bottom-up table filling.

Q: What is a segment tree?
A: A tree of interval summaries supporting range queries and updates, often in O(log n).

Q: What is amortized analysis?
A: A bound on average cost per operation over a sequence, not a probabilistic average.

24. Practice Set and Projects

24.1 Beginner practice

- Find min/max and second-largest element.
- Reverse an array and a linked list.
- Check balanced brackets with a stack.
- Implement queue using two stacks.
- Remove duplicates while preserving order.
- Binary search and find first/last occurrence.

24.2 Intermediate practice

- Detect linked-list cycle and find its entry.
- Merge two sorted linked lists.
- Top-k elements using a heap.
- Group anagrams using a hash map.
- Level-order traversal and tree height.

- BFS shortest path in an unweighted graph.
- Validate a BST.
- Merge overlapping intervals.

24.3 Advanced practice

- Implement LRU cache using map + doubly linked list.
- Range sum queries with Fenwick tree.
- Range minimum queries with segment tree.
- Topological sort and cycle detection.
- Minimum spanning tree using Kruskal + DSU.
- Longest increasing subsequence with O(n log n) method.
- 0/1 knapsack and longest common subsequence DP.
- Trie-based autocomplete with prefix counts.

24.4 Suggested mini-projects

Project	Requirements / learning goal
Contact directory	Hash map for exact lookup; optional trie for prefix search
Task scheduler	Priority queue, priorities, cancellation, tie-breaking
Route planner	Graph representation, BFS/Dijkstra, path reconstruction
Text editor undo	Stack or command history; optionally persistent states
Leaderboard	Hash map plus ordered tree/heap depending on required queries
Range analytics	Prefix sums, Fenwick tree, or segment tree for updates and queries

Final revision checklist

- Can I explain the structure, its invariant, and when to use it?
- Can I implement core operations without looking at notes?
- Can I justify time and space complexity?
- Can I identify edge cases before coding?
- Can I compare two possible structures and explain the trade-off?

End of notes. Keep practicing by implementing structures from scratch, then use standard-library implementations in production code when appropriate.